

Assignment 7: Proximity Search API

Rahul Raj (12341680)

October 9, 2026

API link	<code>http://10.1.75.53:3270/search/?lat=&long=&cat=&rad=</code>
Example	<code>http://10.1.75.53:3270/search/?lat=0.74&long=0.6&cat=bank&rad=0.1</code>

1 Problem

Given a position (`lat`, `long`), a category `cat` and a radius `rad`, return the IDs of the 10 closest locations of that category. The assignment uses two different distances:

- **Radius:** a location is valid only if its straight-line (Euclidean) distance from the query point is at most `rad`.
- **Closeness:** valid locations are ranked by the distance actually travelled on the grid. Links exist only between neighbouring points (no diagonals), and some links are missing.

2 Data

- `locations.csv`: 10,000 locations forming an exact 100×100 grid on $[0, 1]^2$ with spacing $1/99$, and 8 categories with 1,250 locations each. The ID of the point at row r and column c is $100r + c + 1$, where $r = 99 \cdot \text{lat}$ and $c = 99 \cdot \text{long}$.
- `link.txt`: 14,800 lines of the form `long_A lat_A long_B lat_B`. Each line joins two neighbouring grid points. 19,800 neighbour links are possible, so 5,000 (25%) are missing.
- Every link appears only once, always with the smaller point first, so links are **two-way**. As an undirected graph the network is connected: every location can be reached.

3 Choice of distance

Because links are missing, the straight-line distance and the Manhattan distance can both be wrong about which places are close. The distance that matches “actual traversal distance on grid” is the **shortest path in the link graph**. Every link has the same length, so this is the smallest number of links on a path, which breadth-first search (BFS) computes. This is the same idea a ride-sharing app uses: it takes the drivers within some radius of the rider, then ranks them by driving distance on the road network, not by straight-line distance.

Distance	Why it was or was not used
Euclidean	used only for the radius check; it ignores the links
Manhattan	correct only if no link on the way is missing
Shortest path, one-way links	does not match the data (most points become unreachable)
Shortest path, two-way links	used for ranking

Evidence. I generated 2,000 random queries with at least 10 valid answers each, and computed the correct answer for each with an independent Dijkstra search. I then scored each ranking the same way the assignment does: one point per correct ID, out of 100 for 10 queries. IDs tied with the correct 10th also count, since ties are accepted.

Ranking	Expected score / 100	Queries fully correct	Worst query
Road network (this API)	100.0	100.0%	10/10
Euclidean	89.6	30.9%	4/10
Manhattan	89.0	28.9%	3/10
One-way roads	47.1	4.0%	0/10

Table 1: Simulated marks for each way of ranking (2,000 random queries).

Euclidean and Manhattan ranking would get the full list right in less than a third of the queries.

Example. In Figure 1, the grid point nearest to the query has only one link, and it goes south. The banks just north of the query are close in Manhattan distance but far by the links, so the correct answer uses banks to the south and east instead. For this query Manhattan ranking gets only 2 of the 10 IDs right.

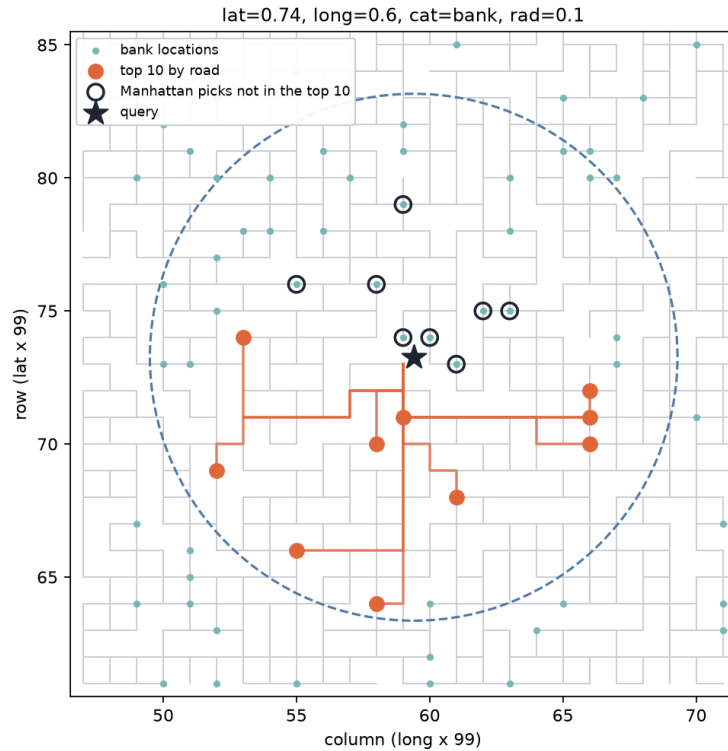


Figure 1: Query $\text{lat}=0.74$, $\text{long}=0.6$, $\text{cat}=\text{bank}$, $\text{rad}=0.1$. Grey lines are links (gaps are missing links); orange shows the 10 results and the paths to them; circled points are what Manhattan ranking would return instead.

4 Algorithm

1. **Start point.** Snap the query to its nearest grid point s by rounding $99 \cdot \text{lat}$ and $99 \cdot \text{long}$. Points outside the square are moved to its border.

2. **Count.** Count the locations of the category inside the circle (one numpy operation). Let $T = \min(10, \text{count})$.
3. **BFS from s .** Each node of the right category within the radius becomes a result. When the T -th result is found at distance d_T , keep going until BFS reaches a node farther than d_T , then stop.
4. **Sort.** Sort the results by (path length, straight-line distance, ID) and return the first 10.

Correctness. BFS visits nodes in increasing distance from s . So once it passes d_T , no node it has not yet visited can be closer than the 10th result. Locations tied with the 10th are all collected before stopping. The circle only filters the results and does not limit the search, because the shortest path to a location inside the circle can pass outside it. Locations that cannot be reached are never returned.

Cost. Each query explores only the area around the query point (about 116 nodes on average), not the whole grid. The graph is built once when the server starts.

Listing 1: The search (app/search.py).

```

55 def search(self, lat, lon, cat, rad, k=10, want_paths=False):
56     start_time = time.perf_counter()
57     net = self.net
58
59     # If fewer than k places are inside the circle we can stop once we have
60     # found all of them, instead of searching the whole map.
61     need = min(k, self.count_in_radius(lat, lon, cat, rad))
62
63     src = net.node_of(lat, lon)
64     dist = {src: 0}
65     parent = {src: -1}
66     queue = deque([src])
67     hits = []
68     cutoff = None # road distance of the need-th hit
69
70     while queue and need > 0:
71         v = queue.popleft()
72         d = dist[v]
73         # BFS pops nodes in order of distance, so once we are past the
74         # distance of the need-th hit nothing closer can turn up. Nodes at
75         # exactly that distance are still collected (ties).
76         if cutoff is not None and d > cutoff:
77             break
78
79         if net.cat[v] == cat:
80             e = math.hypot(net.lat[v] - lat, net.lon[v] - lon)
81             if e <= rad + EPS:
82                 hits.append(Hit(v, net.ids[v], d, e))
83                 if len(hits) == need:
84                     cutoff = d
85
86         # Expand even outside the circle: the shortest road to a place
87         # inside may go out and come back in.
88         for w in net.adj[v]:
89             if w not in dist:
90                 dist[w] = d + 1
91                 parent[w] = v
92                 queue.append(w)
93
94     # closest by road first, then by straight line, then lower ID
95     hits.sort(key=lambda h: (h.hops, h.euclid, h.id))
96     elapsed = (time.perf_counter() - start_time) * 1000

```

```
return SearchResult(src, hits[:k], len(dist), need, elapsed, parent if want_paths else
None)
```

Details.

- **Radius boundary:** a location exactly on the circle counts as inside. A tolerance of 10^{-6} is used because the CSV rounds coordinates to 6 decimals.
- **Category:** spaces are trimmed and case is ignored.
- **Ties:** broken by straight-line distance, then by ID, so results are always the same.

5 API

GET `/search/?lat=&long=&cat=&rad=` returns the 10 IDs, closest first. `/search` without the trailing slash also works. Missing or invalid parameters return HTTP 400 with an error message.

```
$ curl 'http://10.1.75.53:3270/search/?lat=0.74&long=0.6&cat=bank&rad=0.1'
{"ids": [7160, 7059, 6862, 7167, 7267, 7067, 6459, 7454, 6656, 6953]}
```

Adding `&debug=1` also returns, for each result, the path length, the straight-line distance and the path itself.

6 Testing

- On 1,600 random queries (on and off grid points, all categories, radii from 0.02 to 2) the API returns exactly the same ordered list as a brute-force Dijkstra search over all 10,000 points.
- A small 3×3 grid with a missing link checks the exact detour order. Unreachable points are never returned.
- Edge cases: a point exactly on the circle, the corners, points outside the grid, and fewer than 10 valid locations.
- API tests check the route and parameter names, the response format and the errors for bad input. All 21 tests pass.
- A query takes about 0.025 ms inside the server. There were no errors in a 5,000-request load test against the deployed API.